School of Engineering and Technology
Department of Computer Science & Engineering
Python Programming Lab-Mini Project

Div & Roll No. Name of Student:	**Class E** **Roll no 88: Sharvari Sampat Pathade** **Roll no 82: Vaishnavi Sanjay Shinde** **Roll no 87: Riddhi Samrat Ganjwe** **Roll no 85: Proneeta Saha** **Roll no 84: Sanskriti Sharad Kolhe**
Project Title:	**ClearPath: Siren-Triggered Speed Bump Simulation for Ambulances**
Project Description:	**ClearPath is a Python simulation that studies how much time an ambulance loses at a speed bump, and how much a siren-triggered retractable bump could save. A virtual ambulance or car drives toward a virtual speed bump. A simulated detector produces a siren probability every 0.5 seconds. When the probability stays at or above 0.8 for 3 readings in a row, the bump lowers to flat road in 1.5 seconds. Otherwise the bump stays raised, which is the fail-safe default.** **All the logic is written in Python: the vehicle motion, the detector rules, the bump height and the evaluation of each run. The program reports whether the outcome was correct, late, missed or a false alarm, and the time saved compared with a bump that never lowers.** **This is a simulation only. It does not use a microphone or real sound detection, it has not been tested on real roads, and all speeds and timings are assumptions.**
Software used (Frontend & Backend)	**Backend: Python 3 (simulation, detector rules, evaluation and drawing)** **Frontend: Streamlit web page with controls, animation display and results** **Libraries: Streamlit, plus the Python standard library (dataclasses, random, base64, time)**
Application Area:	• **Studying how speed bumps delay emergency vehicles.** • **Exploring how much warning a siren-triggered road feature needs.** • **Comparing correct, late, missed and false-alarm outcomes of a detector.** • **Teaching simulation, state-machine logic and fail-safe design in Python.** • **Planning future work with a trained siren detector and measured speed data.**
Results/Input-Output Screenshots:	**1. Inputs and outputs** **Inputs: sound near the microphone (Siren, Car horn or Silence), vehicle (Ambulance or Normal car), and how far before the bump the siren is first heard**

School of Engineering and Technology
Department of Computer Science & Engineering

(10 to 100 m).

Outputs: an animated road scene, the siren probability, the bump state, a result message, the time saved, and session counters.

All values below are simulation outputs under assumed parameters (60 km/h approach, 4 m/s over a raised bump, 1.5 s for the bump to go flat). With the bump always raised, the simulated ambulance loses about 6 s on this road compared with free-flow driving.

2. Example outputs from the simulation

Input	Output	Time saved
Ambulance, siren heard 100 m before bump	Correct (bump flat in time)	5.8 s
Ambulance, siren heard 80 m before bump	Correct (bump flat in time)	5.1 s
Ambulance, siren heard 60 m before bump	Correct (bump flat in time)	4.1 s
Ambulance, siren heard 40 m before bump	Correct (bump flat in time)	1.9 s
Ambulance, siren heard 20 m before bump	Late (bump still lowering)	0.4 s
Ambulance, car horn only	Missed (bump stayed raised)	0.0 s
Ambulance, silence	Missed (bump stayed raised)	0.0 s
Normal car, silence	Correct (bump stayed raised)	n/a
Normal car, siren playing	False alarm (bump lowered wrongly)	n/a

3. Output frames

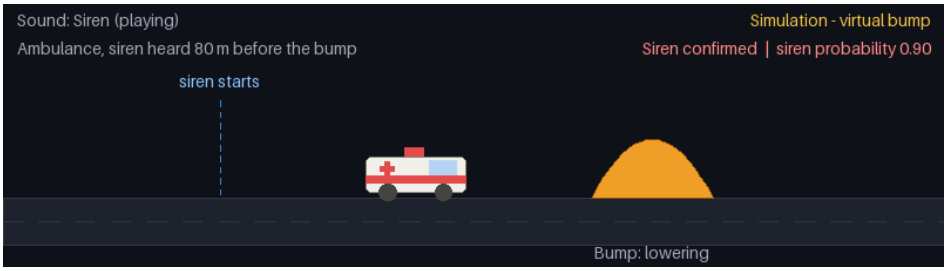


Figure 1: Siren confirmed, bump starting to lower (siren heard 80 m before the bump)

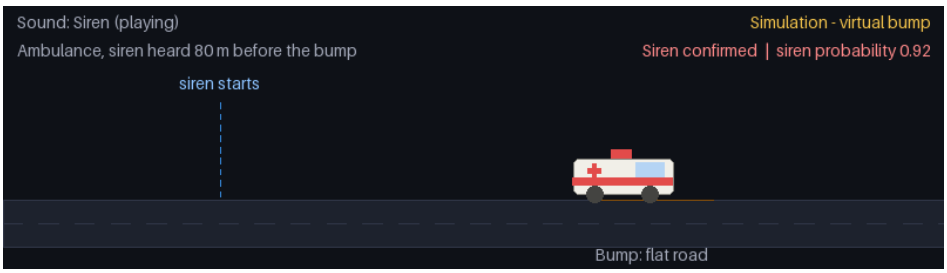


Figure 2: Ambulance crosses with the bump flat (siren heard 80 m before the bump)

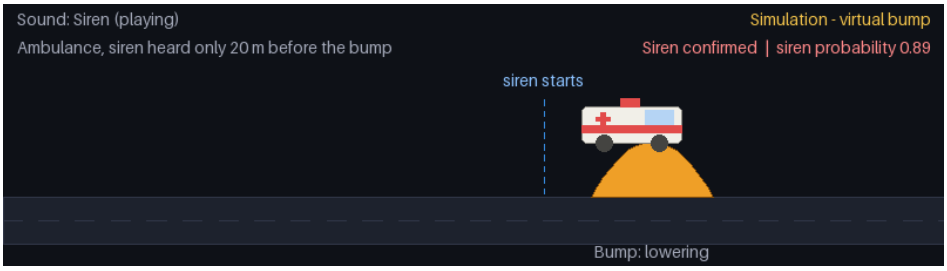


Figure 3: Siren heard too late, bump still lowering when the ambulance arrives



Figure 4: Normal car with no siren, bump stays raised

4. Source code (app.py)

```
"""ClearPath - one-page simulation. Python does all the logic.

A virtual ambulance or car drives toward a virtual speed bump. A simulated
siren detector produces a probability every half second. When the probability
stays high for several readings in a row, the bump lowers into flat road.
Anything else leaves the bump raised (the fail-safe default).

SIMULATION ONLY: the sound choices stand in for a detector. This page does not
use a microphone, does not detect real sounds, and makes no real-world claims.
Every number in PARAMS is an ASSUMPTION, not a measurement.
"""
from __future__ import annotations

import base64
import random
import time
from dataclasses import dataclass
from typing import List, Optional

import streamlit as st


# ---------------------------------------------------------------------------
# 1. Assumptions (ASSUMED - replace with measured data)
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class Params:
    road_length_m: float = 160.0     # length of the virtual road
    bump_pos_m: float = 110.0        # where the bump sits on the road
    cruise_mps: float = 16.7         # ambulance speed, about 60 km/h
    cross_mps: float = 4.0           # speed when crossing a raised bump
    decel_mps2: float = 2.5          # braking
    accel_mps2: float = 1.5          # speeding up again
    threshold: float = 0.8           # probability needed to count as "siren"
    confirm_readings: int = 3        # readings in a row needed to confirm
    reading_period_s: float = 0.5    # one detector reading every 0.5 s
    retract_time_s: float = 1.5      # time for the bump to go fully flat
    hold_time_s: float = 3.0         # bump rises this long after siren stops
    dt: float = 0.05                 # simulation time step


PARAMS = Params()

# What the simulated detector "hears" for each sound (illustrative targets,
# NOT the output of a trained model).
```

School of Engineering and Technology
Department of Computer Science & Engineering

```
SOUND_TARGET = {"Siren": 0.93, "Car horn": 0.22, "Silence": 0.03}
SOUNDS = list(SOUND_TARGET.keys())
VEHICLES = ["Ambulance", "Normal car"]

FLAT_H = 0.02          # bump height (0..1) at or below this counts as flat
RENDER_EVERY = 2       # draw every 2nd simulation step
FRAME_SLEEP_S = 0.06   # pause between drawn frames


# ---------------------------------------------------------------------------
# 2. Simulation (pure Python, no Streamlit)
# ---------------------------------------------------------------------------
@dataclass
class Frame:
    t: float           # time (s)
    x: float           # vehicle position along the road (m)
    v: float           # vehicle speed (m/s)
    prob: float        # simulated siren probability (0..1)
    h: float           # bump height: 1 = raised, 0 = flat road
    confirmed: bool    # True once the siren is confirmed
    siren_on: bool     # True while a siren is playing


def simulate(sound: str, siren_distance_m: float, params: Params = PARAMS,
             seed: int = 42) -> List[Frame]:
    """Run one trip down the road and return every time step.

    sound: "Siren", "Car horn" or "Silence".
    siren_distance_m: for "Siren", how far before the bump it is first heard.
    """
    rng = random.Random(seed)            # fixed seed -> repeatable results
    t, x, v = 0.0, 0.0, params.cruise_mps
    prob = SOUND_TARGET["Silence"]
    streak = 0                           # readings in a row above threshold
    last_positive = -1e9
    confirmed = False
    h = 1.0                              # bump starts raised (fail-safe)
    next_reading = 0.0
    siren_start_x = params.bump_pos_m - siren_distance_m
    frames: List[Frame] = []

    for _ in range(20000):               # safety limit against endless loops
        if x >= params.road_length_m:
            break

        # What is the microphone hearing right now?
        siren_on = (sound == "Siren") and (x >= siren_start_x)
        if sound == "Siren":
            target = SOUND_TARGET["Siren"] if siren_on else SOUND_TARGET["Silence"]
        else:
            target = SOUND_TARGET[sound]

        # Detector: one reading every reading_period_s.
        if t >= next_reading - 1e-9:
            prob = 0.5 * prob + 0.5 * target + rng.uniform(-0.02, 0.02)
            prob = max(0.0, min(1.0, prob))
            if prob >= params.threshold:
                streak += 1
                last_positive = t
            else:
                streak = 0
            if not confirmed and streak >= params.confirm_readings:
                confirmed = True
            if confirmed and (t - last_positive) > params.hold_time_s:
                confirmed = False
            next_reading += params.reading_period_s

        # Bump: lowers while confirmed, rises otherwise (takes retract_time_s).
        step = params.dt / params.retract_time_s
        h = max(0.0, h - step) if confirmed else min(1.0, h + step)

        # Vehicle: slows for the bump unless it is already flat.
        dist = params.bump_pos_m - x
        target_v = params.cruise_mps
        if h > FLAT_H and dist >= -4.0:
            brake_dist = max(0.0, (v * v - params.cross_mps ** 2) / (2 *
params.decel_mps2)) + 3.0
```

School of Engineering and Technology
Department of Computer Science & Engineering

```
            if dist <= brake_dist:
                target_v = params.cross_mps
        if v > target_v:
            v = max(target_v, v - params.decel_mps2 * params.dt)
        else:
            v = min(target_v, v + params.accel_mps2 * params.dt)

        frames.append(Frame(t, x, v, prob, h, confirmed, siren_on))
        x += v * params.dt
        t += params.dt

    return frames


@dataclass
class Outcome:
    kind: str                   # correct, late, missed or false_alarm
    trip_s: float               # time to drive the whole road
    saved_s: float              # time saved compared with a raised bump
    detect_delay_s: Optional[float]   # siren start -> confirmed (None if never)


def evaluate(vehicle: str, frames: List[Frame], baseline: List[Frame],
             params: Params = PARAMS) -> Outcome:
    """Decide what happened when the vehicle reached the bump."""
    at_bump = next((f for f in frames if f.x >= params.bump_pos_m), frames[-1])
    is_flat = at_bump.h <= FLAT_H

    if vehicle == "Ambulance":
        if is_flat:
            kind = "correct"
        elif at_bump.confirmed:
            kind = "late"          # confirmed, but bump still lowering
        else:
            kind = "missed"        # never confirmed in time
    else:
        kind = "false_alarm" if at_bump.confirmed else "correct"

    t_on = next((f.t for f in frames if f.siren_on), None)
    t_conf = next((f.t for f in frames if f.confirmed), None)
    delay = (t_conf - t_on) if (t_on is not None and t_conf is not None) else None

    trip = frames[-1].t
    return Outcome(kind, trip, baseline[-1].t - trip, delay)


# ---------------------------------------------------------------------------
# 3. Drawing (Python builds the picture as SVG text)
# ---------------------------------------------------------------------------
SCALE, OFFSET = 3.9, 40.0      # metres -> pixels


def px(x_m: float) -> float:
    return OFFSET + x_m * SCALE


def vehicle_shapes(vehicle: str) -> str:
    if vehicle == "Normal car":
        return (
            '<path d="M0 -6 L0 -16 L14 -18 L24 -30 L50 -30 L60 -18 L72 -16 L72 -6 Z" '
            'fill="#888780" stroke="#444441" stroke-width="1"/>'
            '<path d="M26 -18 L30 -27 L38 -27 L38 -18 Z M42 -18 L42 -27 L48 -27 L55 -18
Z" fill="#B5D4F4"/>'
            '<circle cx="16" cy="-4" r="6" fill="#444441"/><circle cx="56" cy="-4" r="6"
fill="#444441"/>'
        )
    return (
        '<rect x="0" y="-30" width="72" height="26" rx="4" fill="#f1efe8"
stroke="#888780" stroke-width="1"/>'
        '<rect x="0" y="-16" width="72" height="5" fill="#E24B4A"/>'
        '<rect x="46" y="-27" width="20" height="10" rx="2" fill="#B5D4F4"/>'
        '<rect x="14" y="-26" width="3" height="10" fill="#E24B4A"/>'
        '<rect x="10.5" y="-22.5" width="10" height="3" fill="#E24B4A"/>'
        '<rect x="28" y="-36" width="14" height="6" rx="2" fill="#E24B4A"/>'
        '<circle cx="16" cy="-4" r="6" fill="#444441"/><circle cx="56" cy="-4" r="6"
fill="#444441"/>'
    )
```

School of Engineering and Technology
Department of Computer Science & Engineering

```
def bump_label(f: Frame) -> str:
    if f.h <= FLAT_H:
        return "Bump: flat road"
    if f.confirmed:
        return "Bump: lowering"
    if f.h < 0.98:
        return "Bump: rising"
    return "Bump: raised"


def scene_svg(f: Frame, vehicle: str, sound: str, siren_distance_m: float,
              params: Params = PARAMS) -> str:
    """Return one picture of the road, bump and vehicle."""
    cx = px(f.x)
    bx = px(params.bump_pos_m)
    half = 44.0

    # Vehicle rides up and over the bump while the bump is not flat.
    d = (cx - bx) / half
    lift = 0.0
    if f.h > FLAT_H and abs(d) < 1:
        lift = half * f.h * (1 - d * d)

    peak_ctrl = 140 - 2 * half * f.h
    marker = ""
    if sound == "Siren":
        mx = px(params.bump_pos_m - siren_distance_m)
        marker = (
            f'<line x1="{mx:.1f}" y1="70" x2="{mx:.1f}" y2="140" stroke="#378ADD" '
            'stroke-width="1" stroke-dasharray="4 4"/>'
            f'<text x="{mx:.1f}" y="64" text-anchor="middle" font-size="11"
fill="#8cc2ff">siren starts</text>'
        )

    sound_text = f"Sound: {sound}" + (" (playing)" if f.siren_on else "")
    return (
        '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 680 190" width="680"
height="190">'
        '<rect x="0" y="140" width="680" height="34" fill="#1d222d" stroke="#313748"
stroke-width="0.5"/>'
        '<line x1="0" y1="157" x2="680" y2="157" stroke="#313748" stroke-width="1"
stroke-dasharray="14 12"/>'
        f'{marker}'
        f'<path d="M{bx - half:.1f} 140 Q{bx:.1f} {peak_ctrl:.1f} {bx + half:.1f} 140 Z"
'
        'fill="#EF9F27" stroke="#854F0B" stroke-width="1"/>'
        f'<g transform="translate({cx - 36:.1f},{140 -
lift:.1f})">{vehicle_shapes(vehicle)}</g>'
        f'<text x="{bx:.1f}" y="186" text-anchor="middle" font-size="12"
fill="#a3a8b8">{bump_label(f)}</text>'
        f'<text x="10" y="20" font-size="12" fill="#a3a8b8">{sound_text}</text>'
        '</svg>'
    )


def svg_to_html(svg: str) -> str:
    """Wrap SVG text so Streamlit can show it."""
    b64 = base64.b64encode(svg.encode("utf-8")).decode("ascii")
    return f'<img style="width:100%" src="{b64}"/>'


def badge(text: str, bg: str, fg: str) -> str:
    return (f'<span style="font-size:12px;padding:3px 10px;border-radius:8px;'
            f'background:{bg};color:{fg}">{text}</span>')


# ---------------------------------------------------------------------------
# 4. Messages
# ---------------------------------------------------------------------------
def describe(vehicle: str, outcome: Outcome, params: Params = PARAMS) -> tuple[str,
str]:
    """Return (level, text) for the result message."""
    saved = f"Time saved compared with a raised bump: {outcome.saved_s:.1f} s (assumed
speeds)."
    if outcome.kind == "false_alarm":
```

School of Engineering and Technology
Department of Computer Science & Engineering

```
        return "error", "False alarm: the bump lowered for a normal car."
    if vehicle == "Normal car":
        return "success", "Normal car: no siren was confirmed, so the bump stayed
raised, as it should."
    if outcome.kind == "correct":
        return "success", f"Ambulance crossed with the bump flat in time. {saved}"
    if outcome.kind == "late":
        return "warning", f"Siren confirmed too late: the bump was still lowering when
the ambulance arrived. {saved}"
    return "warning", "Siren not confirmed in time. The bump stayed raised and the
ambulance slowed over it."


def timing_note(outcome: Outcome, params: Params = PARAMS) -> Optional[str]:
    if outcome.detect_delay_s is None:
        return None
    need_s = outcome.detect_delay_s + params.retract_time_s
    need_m = params.cruise_mps * need_s
    return (f"Siren confirmed {outcome.detect_delay_s:.1f} s after it started, and the
bump needs "
            f"{params.retract_time_s:.1f} s more to go flat. At {params.cruise_mps *
3.6:.0f} km/h "
            f"that needs about {need_m:.0f} m of warning (assumed values).")


# ---------------------------------------------------------------------------
# 5. Page (Streamlit only shows what Python computed)
# ---------------------------------------------------------------------------
HIDE_CSS = """
<style>
  [data-testid="stSidebar"], [data-testid="stSidebarNav"],
  [data-testid="collapsedControl"], [data-testid="stToolbar"],
  [data-testid="stDecoration"], header[data-testid="stHeader"],
  #MainMenu, footer { display: none !important; }
  .block-container { max-width: 1100px; padding-top: 1.5rem; padding-bottom: 1rem; }
</style>
"""


def main() -> None:
    st.set_page_config(page_title="ClearPath live demo", layout="wide",
                       initial_sidebar_state="collapsed")
    st.markdown(HIDE_CSS, unsafe_allow_html=True)

    if "counts" not in st.session_state:
        st.session_state.counts = {"runs": 0, "correct": 0, "missed": 0, "false_alarm":
0}
    if "last_result" not in st.session_state:
        st.session_state.last_result = None

    head_left, head_right = st.columns([2, 3])
    head_left.markdown('<span style="font-size:20px;font-weight:600">ClearPath live
demo</span>',
                       unsafe_allow_html=True)
    badge_ph = head_right.empty()
    waiting = (badge("Simulation - virtual bump", "#3a2f12", "#f0c14b") + "   " +
               badge("Waiting", "#262b38", "#a3a8b8"))
    badge_ph.markdown(f'<div style="text-align:right">{waiting}</div>',
unsafe_allow_html=True)

    scene_ph = st.empty()

    c1, c2, c3 = st.columns([1, 1, 2])
    sound = c1.radio("1. Sound near the microphone", SOUNDS)
    vehicle = c2.radio("2. Vehicle", VEHICLES)
    distance = c3.slider("3. Siren is first heard this far before the bump (m)",
                         10, 100, 80, step=5, disabled=(sound != "Siren"))
    run = st.button("Run simulation", type="primary")

    prob_label_ph = st.empty()
    prob_ph = st.empty()

    idle = Frame(0.0, 0.0, PARAMS.cruise_mps, SOUND_TARGET["Silence"], 1.0, False,
False)
    scene_ph.markdown(svg_to_html(scene_svg(idle, vehicle, sound, distance)),
unsafe_allow_html=True)
    prob_label_ph.caption("Siren probability (illustrative, not a trained model): 0.03")
```

School of Engineering and Technology
Department of Computer Science & Engineering

```
    prob_ph.progress(3)

    if run:
        frames = simulate(sound, distance)
        baseline = simulate("Silence", distance)   # same road, bump never lowers
        for i, f in enumerate(frames):
            if i % RENDER_EVERY == 0 or i == len(frames) - 1:
                scene_ph.markdown(svg_to_html(scene_svg(f, vehicle, sound, distance)),
                                  unsafe_allow_html=True)
                prob_label_ph.caption(
                    f"Siren probability (illustrative, not a trained model):
{f.prob:.2f}")
                prob_ph.progress(int(f.prob * 100))
                state = (badge("Siren confirmed", "#3b1a1a", "#ff8a8a") if f.confirmed
                         else badge("Waiting", "#262b38", "#a3a8b8"))
                badge_ph.markdown(
                    '<div style="text-align:right">'
                    + badge("Simulation - virtual bump", "#3a2f12", "#f0c14b")
                    + "   " + state + "</div>", unsafe_allow_html=True)
                time.sleep(FRAME_SLEEP_S)

        outcome = evaluate(vehicle, frames, baseline)
        counts = st.session_state.counts
        counts["runs"] += 1
        if outcome.kind == "correct":
            counts["correct"] += 1
        elif outcome.kind == "false_alarm":
            counts["false_alarm"] += 1
        else:
            counts["missed"] += 1
        st.session_state.last_result = (describe(vehicle, outcome),
timing_note(outcome))

    counts = st.session_state.counts
    m1, m2, m3, m4 = st.columns(4)
    m1.metric("Runs", counts["runs"])
    m2.metric("Correct", counts["correct"])
    m3.metric("Missed or late", counts["missed"])
    m4.metric("False alarm", counts["false_alarm"])

    if st.session_state.last_result is None:
        st.info("Choose a sound and a vehicle, then click Run simulation.")
    else:
        (level, text), note = st.session_state.last_result
        getattr(st, level)(text)
        if note:
            st.caption(note)

    st.caption("Simulation only. No microphone or real detection is used, and the
numbers "
               "are assumptions, not measurements. Counts are for this session and are "
               "not a measured accuracy.")


if __name__ == "__main__":
    main()
```

Name and Signature of Students **Name and Signature of Instructor**